5.

Explain in your own words the difference between encapsulation and abstraction, using examples from any social media app you use daily (like Instagram or WhatsApp).

ANS:

Encapsulation is about **hiding the internal data and only exposing controlled ways to interact with it** — it's a *data-protection* concept.

Instagram example: Your account's password and phoneNumber are private data. You can't reach into Instagram's database and edit them directly. Instead, Instagram gives you a "Change Password" screen (a public method, essentially) that validates your input before updating the real, hidden value. You never touch the raw data — you go through the app's controlled interface. That's encapsulation: bundling data with the only methods allowed to touch it, and keeping the raw data private.

Abstraction is about **hiding *how* something works internally and only showing *what* it does** — it's a *complexity-hiding* concept, focused on interface over data.

WhatsApp example: When you tap "Send" on a message, you don't see (or need to know) how WhatsApp encrypts your message, routes it through servers, handles delivery receipts, or retries on poor network. All of that complex logic is hidden. You just interact with a simple action: sendMessage(). That's abstraction — exposing a simple interface ("send this message") while hiding the messy implementation details behind it.

The key difference:

	Encapsulation	**Abstraction**
Focus	**Protecting data** (who can access/modify it)	**Hiding complexity** (how something works internally)
Achieved via	private/protected fields + public getters/setters	Abstract classes/interfaces, exposing only essential methods
Question it answers	"Can you touch this directly?"	"Do you need to know how this works?"
App example	You can't directly edit your stored password — only through a validated "Change Password" flow	You don't know (or care) how Instagram's algorithm decides your feed order — you just see the feed